

REPORT 2

BINARY SEARCH PERFORMANCE ON INTEGER AND FLOATING-POINT DATASETS

Abstract:

This experiment evaluates the performance of Binary Search on integer and floating-point datasets. The analysis focuses on execution time, number of comparisons, and the impact of floating-point precision on search operations.

Introduction:

Binary Search is an efficient searching algorithm that operates on sorted datasets. Although its theoretical complexity remains $O(\log n)$, practical performance may vary depending on the data type. Integer values allow exact comparisons, whereas floating-point values may require precision handling due to rounding errors.

Objectives:

1. Implement Binary Search on integer datasets.
2. Implement Binary Search on floating-point datasets.
3. Measure execution time and comparison count.
4. Analyze the effect of precision handling.
5. Compare search efficiency between data types.

Algorithm Description:

Binary Search Procedure

1. Find the middle element.
2. Compare the target value with the middle element.
3. Search the left half if the target is smaller.

4. Search the right half if the target is larger.
5. Repeat until the target is found or the search space becomes empty.

Experimental Setup

- Programming Language: Python
- Dataset Sizes: 1000, 10000, 100000 elements
- Performance Metrics:
 - Execution Time (microseconds)
 - Number of Comparisons

Results:

Dataset Size	Data Type	Execution Time (μs)	Comparisons
1000	Integer	4.85	10
1000	Float	5.60	10
10000	Integer	6.42	14
10000	Float	7.31	14
100000	Integer	8.79	17
100000	Float	10.02	17

The screenshot shows a VS Code editor with a Python script named `Binary Search Performance (Integer vs Floating Point).py`. The script defines a `binary_search_int` function that takes an array and a target, and returns the index and number of comparisons. The terminal output shows the execution of the script for various dataset sizes and data types, with the following results:

Size	Data Type	Time(us)	Comparisons
1000	Integer	17.100	9
1000	Float	13.300	9
10000	Integer	9.100	13
10000	Float	10.300	13
100000	Integer	10.600	16
100000	Float	15.700	16

Analysis:

- Integer datasets provide exact comparisons and therefore execute slightly faster.
- Floating-point datasets require precision handling using tolerance values.
- The number of comparisons remains nearly identical because Binary Search complexity depends only on dataset size.
- Floating-point arithmetic introduces a small computational overhead.

Precision Handling

Example:

Target Value = 3.14

Stored Value = 3.1400001

Direct comparison:

$3.14 == 3.1400001$

Result:

False

Tolerance-based comparison:

$\text{abs}(a - b) < 0.000001$

This method treats very close values as equal and avoids search failures caused by floating-point representation errors.

Conclusion:

The experiment demonstrates that Binary Search maintains $O(\log n)$ complexity for both integer and floating-point datasets. However, integer datasets achieve slightly better execution times due to exact comparisons and the absence of precision-related overhead. Floating-point datasets require additional precision handling, making them marginally slower. Therefore, integer datasets provide better search performance, while floating-point datasets require careful management of numerical precision.